

Shadow Revenue Detection System

PySpark | Spark SQL | Medallion Architecture | Databricks

1. Problem Statement

In modern retail and e-commerce systems, large volumes of transactional data are generated across multiple platforms such as order management, payment systems, and product catalogs, often leading to inconsistencies between recorded sales, received payments, and actual pricing. These discrepancies result in hidden financial leakages, known as shadow revenue, which are not easily detected through standard reporting. Challenges such as missing payments, orphan payments, duplicate records, pricing mismatches, and overall data quality issues make it difficult for businesses to accurately measure revenue and maintain data integrity. Therefore, there is a need for a robust data engineering pipeline that can ingest, clean, and transform raw data into a structured format, enabling the identification, quantification, and analysis of these anomalies to support reliable business decision-making. complex retail environments, revenue loss is not always directly visible in standard financial reports. The following five critical inconsistencies are targeted:

Anomaly Type	Description
Missing Payments	Orders created in the system but never matched to a payment record — revenue expected but not received.
Orphan Payments	Payments received in the payment system with no corresponding order — money that cannot be attributed to any sale.
Duplicate Orders	System-generated duplicate order records that artificially inflate reported revenue figures.
Price Mismatches	Transactions billed at a price that diverges from the authoritative product catalog — causing margin erosion.

Date / Time Drift	Transactions recorded on incorrect dates due to delayed processing — causing reconciliation failures.
-------------------	---

2. Description

The **Shadow Revenue Detection System** is an enterprise-grade data engineering and analytics pipeline designed to identify, quantify, and resolve hidden financial leakages within a high-volume retail ecosystem. The system processes disparate transactional datasets using **PySpark** and **Spark SQL** frameworks running on a **Databricks** cluster.

The pipeline is architected around the **Medallion paradigm (Bronze → Silver → Gold)** and produces a unified, actionable **Revenue Integrity Report** for business stakeholders. A comparative framework contrasts a naive (messy) pipeline against an optimised (production-grade) pipeline to quantify the financial impact of poor data engineering practices.

Metric	Value
Total Synthetic Orders Generated	10,000 (+ ~2% duplicates = ~10,200)
Total Synthetic Payments Generated	~9,300 (90% of orders paid)
Orphan Payments Injected	~300 (3% of orders with fake order IDs)
Price Mismatches Injected	~10% of orders
Duplicate Orders Injected	~2% of orders
Output	Revenue Integrity Report — Bad vs Good pipeline comparison

3. Architectural Overview — Medallion Paradigm

Medallion Architecture organises data into three progressive quality layers. Each layer builds on the previous, increasing data quality, reliability, and analytical readiness.

Layer	Role & Technology
Bronze — Raw Ingestion	PySpark DataFrames. Raw datasets (Orders, Payments, Products, Customers) are ingested from source systems in native format and written as Parquet files. Immutable, historical record.
Silver — Cleansing & Enrichment	PySpark SQL Functions & DataFrame API. Deduplication via Window functions, type casting to DecimalType, date parsing, null filtering, SCD Type 2 handling. Data quality enforced.
Gold — Analytics & Reporting	Spark SQL complex joins (Anti-Joins, Outer Joins), aggregations. Orphan payment detection, missing order flagging, price mismatch analysis, KPI computation, Bad vs Good comparison.

Storage Path	Contents
/raw_layer/orders	Raw order transactions (Parquet)
/raw_layer/payments	Raw payment records (Parquet)
/raw_layer/products	Product catalog with SCD Type 2 history (Parquet)
/raw_layer/customers	Customer master data (Parquet)
fact_revenue_bad	Gold table — naive pipeline output (with duplicates, float errors)
fact_revenue_good	Gold table — optimised pipeline output (clean, deduplicated, SCD-aware)
kpi_revenue_bad / kpi_revenue_good	KPI summary tables for Bad and Good pipelines
kpi_comparison	UNION of Bad and Good KPIs for side-by-side dashboard comparison

4. Objectives

- To design and implement an end-to-end data engineering pipeline for processing large-scale retail transaction data.
- To ingest and consolidate data from multiple sources such as orders, payments, products, and customers into a unified system.
- To clean and transform raw data by handling duplicates, missing values, and inconsistent formats to ensure data quality.
- To identify and detect key revenue anomalies including missing payments, orphan payments, duplicate transactions, and price mismatches.
- To accurately compute business KPIs such as total revenue, total payments, revenue difference, and accuracy ratio.
- To compare naive (unprocessed) and optimized (cleaned) pipelines to highlight the impact of data engineering on financial accuracy.
- To build a structured Medallion Architecture (Bronze, Silver, Gold) for scalable and maintainable data processing.
- To generate a final Revenue Integrity Report that provides actionable insights for business stakeholders.

5. Datasets

The Shadow Revenue Detection System utilizes four primary datasets representing different aspects of a retail ecosystem:

- **Orders Dataset:** Contains transaction-level details such as order ID, product ID, customer ID, quantity, price, order date, and order status. This dataset may include duplicate records and price inconsistencies.
- **Payments Dataset:** Stores payment-related information including order ID, payment amount, payment method, and payment date. It may contain missing payments (no payment for an order) and orphan payments (payments without a corresponding order).
- **Products Dataset (SCD Type 2):** Maintains product catalog information with historical pricing using Slowly Changing Dimension Type 2. It includes product ID, price, effective dates, and a flag indicating the current active record.
- **Customers Dataset:** Contains customer-related information such as customer ID, name, city, and signup date, providing context for transactional analysis.

5. KPI Derivation Logic

The system derives key performance indicators (KPIs) from the cleaned and processed data in the Gold layer:

- **Total Revenue:** Calculated as the sum of (quantity × product_price) using the latest product catalog price to ensure accuracy.
- **Total Payment:** Sum of all payment amounts recorded in the payments dataset.
- **Revenue Difference:** The difference between total revenue and total payment, representing potential financial leakage.
- **Accuracy Ratio:** Ratio of total payments to total revenue, indicating how much of the expected revenue has been successfully realized.
- **Missing Payments Count:** Number of orders with no corresponding payment record (identified using NULL values after left join).
- **Orphan Payments Count:** Number of payments that do not match any order (identified using left anti join).
- **Price Mismatch Count:** Number of transactions where the order price differs from the product catalog price.

These KPIs enable a comprehensive understanding of revenue discrepancies and data quality issues.

6. Expected Dashboard Visuals

The final dashboard presents key insights derived from the Gold layer to help stakeholders understand revenue integrity and detect anomalies effectively.

- **KPI Cards (Top Summary Metrics)**
Displays high-level metrics such as Total Revenue, Total Payment, Revenue Difference, Accuracy Ratio, and total counts of Missing Payments and Orphan Payments. These provide a quick snapshot of overall financial health.
- **Bar Chart — Revenue Comparison (Bad vs Good Pipeline)**
Compares total revenue and total payments between the unprocessed (bad) pipeline and the cleaned (good) pipeline. This highlights how data cleaning impacts financial accuracy.
- **Bar Chart — Missing Payments Count**
Shows the number of orders with no corresponding payments. Helps identify the extent of revenue leakage due to unpaid transactions.

- **KPI Card — Orphan Payments**
Displays the total number of payments that do not match any order. This highlights untraceable revenue in the system.
- **Donut / Pie Chart — Issue Distribution**
Visualizes the proportion of different anomaly types such as Missing Payments, Price Mismatches, and Duplicate Records. Helps identify the dominant source of revenue leakage.
- **Line Chart — Revenue Trend Over Time**
Shows daily or weekly revenue trends, helping stakeholders observe fluctuations, patterns, or anomalies in revenue over time.
- **Table — Per-Order Revenue Difference (Bad vs Good)**
Displays order-level comparison of revenue between the bad and good pipelines, highlighting discrepancies caused by duplicates, mismatches, or incorrect pricing.
- **Bar Chart — Revenue Difference (Leakage)**
Shows the gap between expected revenue and actual payments, giving a direct visualization of financial leakage.

7. Conclusion

The Shadow Revenue Detection System demonstrates how a well-designed data engineering pipeline can transform raw, inconsistent transactional data into reliable and actionable insights. By leveraging the Medallion Architecture and applying robust data cleaning, transformation, and anomaly detection techniques, the system effectively identifies hidden revenue leakages such as missing payments, orphan transactions, and pricing inconsistencies. The comparison between naive and optimized pipelines highlights the critical role of data quality in ensuring accurate financial reporting. Ultimately, this system empowers business stakeholders with a clear and trustworthy view of revenue integrity, enabling better decision-making and improved operational efficiency.